UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

CS2024 - Theory of Computation

Regular Expressions

Angel Napa

TRANSFORMATEC

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

0.

Arithmetic Expression

When things were simpler

TRANSFORMATEC

WHAT is an Arithmetic Expression?

- Explicit definition:
  - Spooky
- Arithmetic Operations:
  - Addition(+),Negation (-),Multiplication(*), etc.
- Examples:
  - 3, 4+5, ((4+4)*4) - 4, 44/44 , 4+6+7 etc.
- Formal definition?

Inductive definition

For the sake of our mind, we're restricting to integer numbers, and operations binary operators **+,*,** and unary operation **-**

We say A is an arithmetic expression if A is:

- n, for some integer n in $\mathbb{Z}$
- (A_1+A_2), where A_1 & A_2 are arithmetic expressions
- (A_1*A_2), where A_1 & A_2 are arithmetic expressions
- $(-A_1)$, where A_1 is an arithmetic expression

Parenthesis can be omitted, if that's the case, we evaluate the operators in precedence order: first (-), then (*), then (+).

- -3+4*5

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

1.

Regular Expression

It was always regular

TRANSFORMATEC

WHAT is a Regular Expression?

- Explicit definition:
  - Spooky
- Regular Operations:
  - Union(U), Concatenation(·), star(*)
- Examples:
  - {0}, {}, {01}·{0,00},{0,1} U {1,10} , {0}* + {1}·{0,101}, etc.
- Formal definition?

Inductive definition

We say R is a Regular Expression if R is:

- ω, for some symbol ω in Σ
- ε
- $\varnothing$
- $(R_1 \cup R_2)$, where R_1 & R_2 are regular expressions
- $(R_1 \cdot R_2)$, where R_1 & R_2 are regular expressions
- (R_1^*), where R_1 is a regular expression

Parenthesis can be omitted, if that's the case, we evaluate the operators in precedence order: first (*), then (·), then (∪).

Inductive definition

Let L(R) be the language represented by R, then:

- $L(\omega) = \{\omega\}$, for some integer ω in Σ
- $L(\varepsilon) = \{\varepsilon\}$
- $L(\varnothing) = \varnothing$
- $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$, where R_1 & R_2 are regular expressions
- $L(R_1 R_2) = L(R_1) \cdot L(R_2)$, where R_1 & R_2 are regular expressions
- $L(R_1^*) = L(R_1)^*$, where R_1 is a regular expression

Parenthesis can be omitted, if that's the case, we evaluate the operators in precedence order: first (*), then (·), then (∪).

Some notations

For a regular expression R, we have:

- $R^2 = R \cdot R$
- $L(R^2) = \{\omega: \omega$ is a concatenation of exactly 2 strings of $R\}$
- $L(R^i) = \{\omega: \omega$ is a concatenation of exactly i strings of $R\}$
- $R^0 = \varepsilon$
- $R^* = R^0 \cup R^1 \cup R^2 \cup R^3 \cup \ldots$
- $R^+ = \quad R^1 \cup R^2 \cup R^3 \cup \ldots$

Example 1

What language represents the regular expression (0 U 1)*1?

- 0 represents the language {0}
- 1 represents the language {1}
- (0 U 1) represents the language {0,1}
- (0 U 1)* represents the language {0,1}*
- (0 U 1)*1 represents the language {0,1}* · 1
- Strings in the language (0 U 1)*1:
  - {1,01,11,001,011,101,111,...}
- Descripcion 1: All strings you can obtain concatenating any strings of 0s and 1s, with a 1 at the end
- Description 2: All strings that end with a 1

Example 2

What language represents the regular expression $(0 \cup 1)0^*$?

- 0^* represents the language $\{0\}^*$
- $(0 \cup 1)0^*$ represents the language $\{0,1\} \cdot \{0\}^*$
- Strings in the language $\{0,1\} \cdot \{0\}^*$:
  - $\{1,0,10,00,100,000,1000,0000...\}$
- Descripcion 1: All strings you can obtain concatenating ona symbol, with a strings of 0s
- Description 2: All strings whose first symbol can be anything, and the rest are all 0s.

Exercise 1

Describe the languages that represent the following RE. $\Sigma = \{0,1\}$

- 0^*10^*
- $\Sigma^*1\Sigma^*$
- $\Sigma^*001(0 \cup 1)^*$
- $(\Sigma\Sigma\Sigma)^*$
- $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1$
- $(0 \cup \varepsilon)(1 \cup \varepsilon)$
- $1^*\varnothing$
- $\varnothing^*$
- $(01^+)^*$

Exercise 2

Build RE for the following languages

- Strings of the alphabet {0,1} with alternating zeros and ones
- Strings of the alphabet {a,b,c} with at least one a, and one b
- Strings of the alphabet {0,1} with fourth symbol 0.
- Strings of the alphabet {0,1} with at most one substring 11
- Strings of the alphabet {0,1} whose number of 0s is divisible by 3

Wait a Moment

- We talked about regular expressions R
- We talked about languages that are represented by a regular expression
- And we talked about regular languages
- Is there a relation between those objects?
  - **YES**

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

2.

Equivalence with DFA

It is the same?

TRANSFORMATEC

Theorem

A language is regular if and only if some regular expression describes it

L is regular if there is some RE R with L = L(R)

Lemma

If R is a regular expression, then L(R) is regular

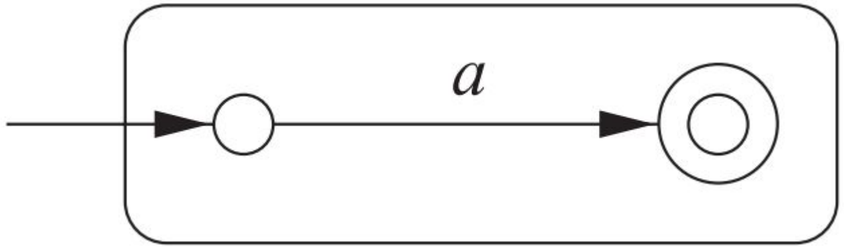
In other words, R describes a language that can be recognized by an NFA -ε

Proof Idea

- We can build inductively a NFA that recognizes L(R)
- We construction is basically what we saw before

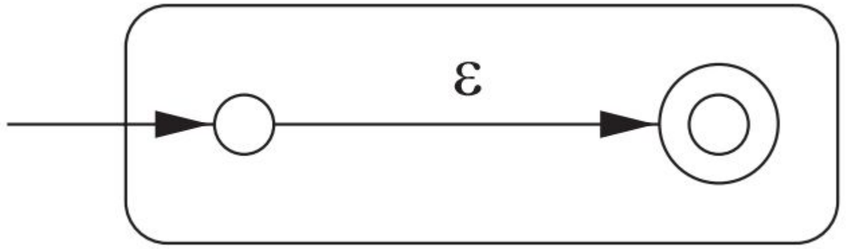
Proof

- R = a, for some a∈Σ. The following NFA recognizes L(R)



Proof

- R = ε. The following NFA recognizes L(R)

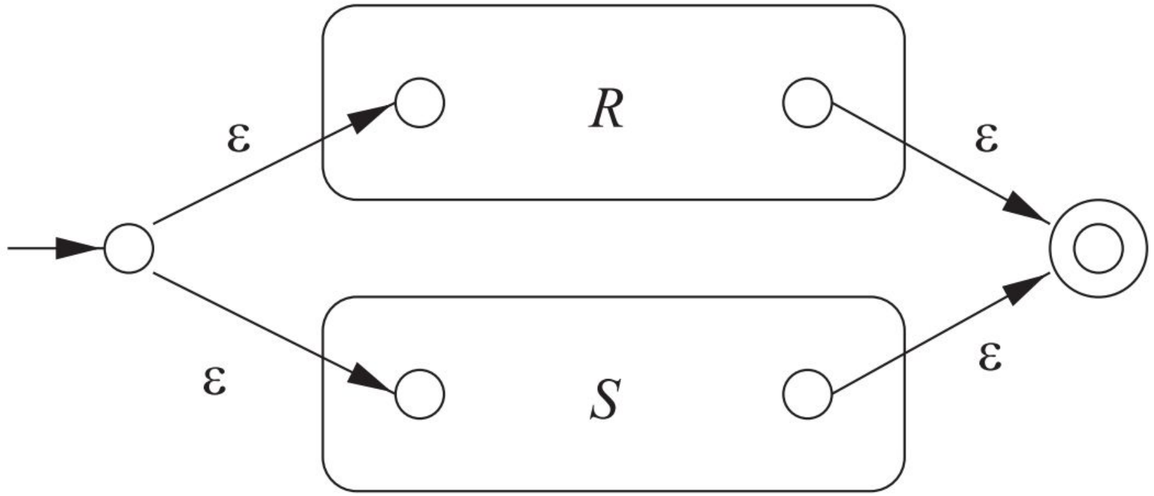


Proof

- R = ⌀. The following NFA recognizes L(R)

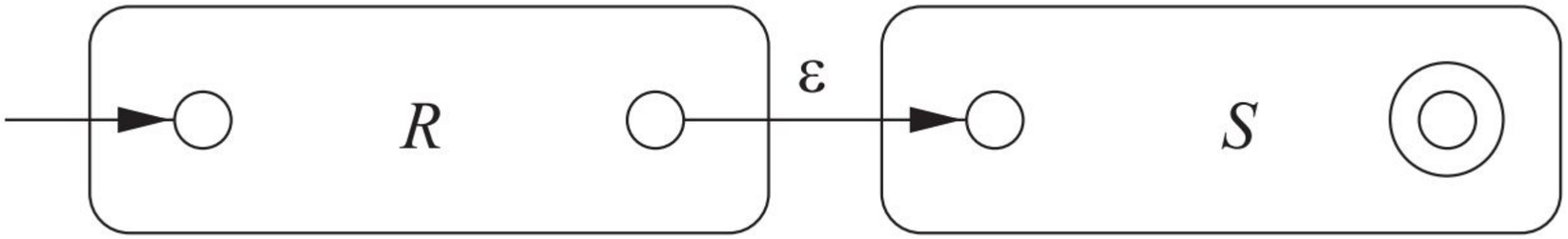
Proof

- T = R ∪ S, for some regular expressions R an S. The following NFA recognizes L(T)



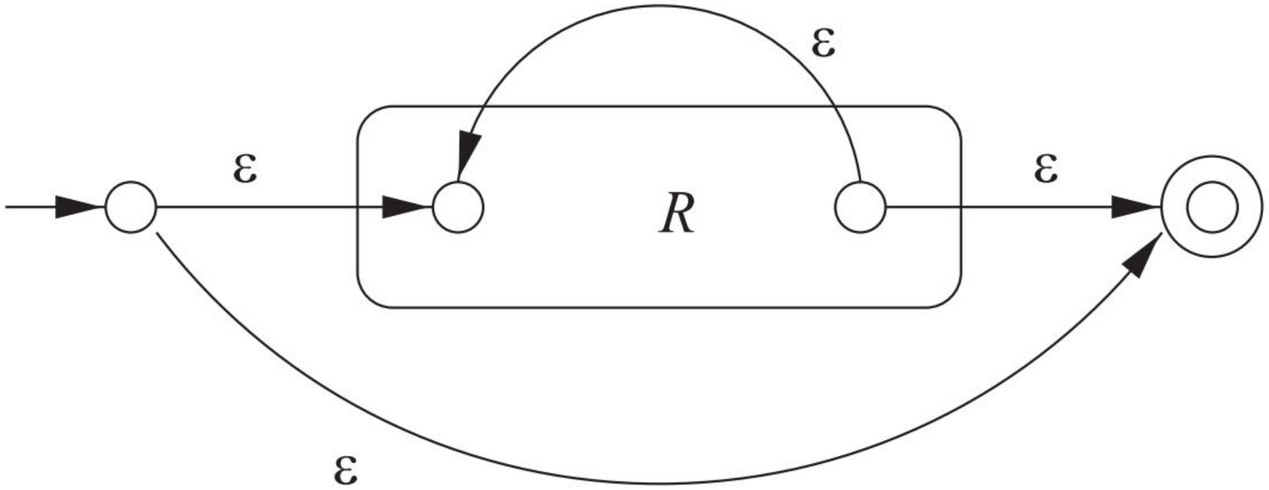
Proof

- T = R·S, for some regular expressions R an S. The following NFA recognizes L(T)



Proof

- T = R*, for some regular expression R. The following NFA recognizes L(T)



Lemma

If L is a regular language, then it is described by a regular expression

In other words, there exists R such that L(R) = L

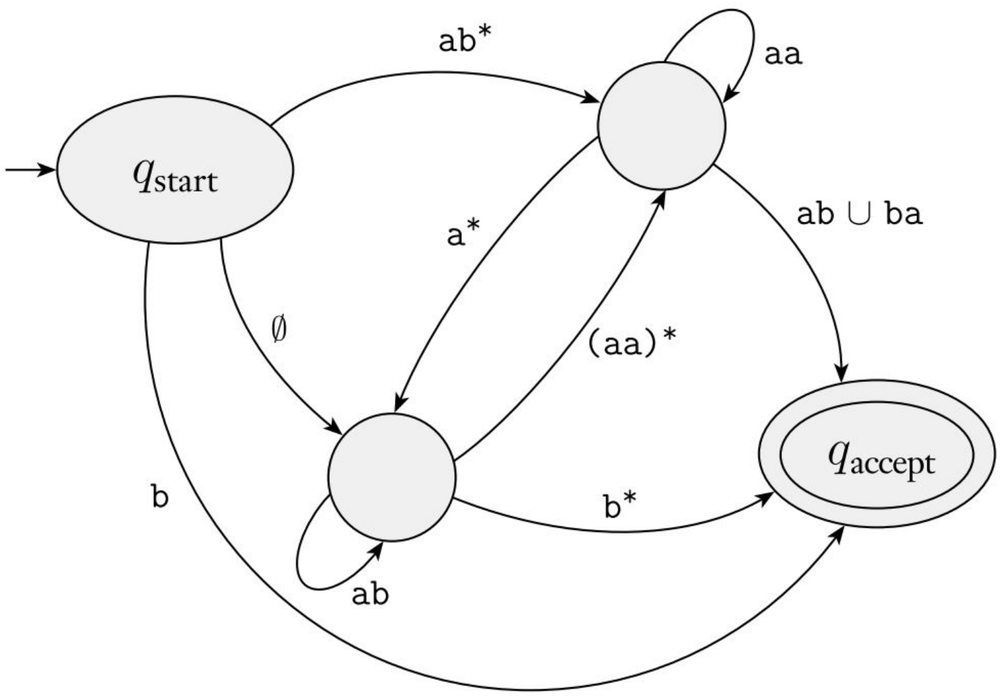
Proof Idea

- If L is regular, there is a DFA A that recognizes L
- convert A to a GNFA B
- Convert B to a regular expression

Generalized NFA (GNFA)

- NFAs whose transitions can have regular expressions
- convert A to a GNFA B
- Unlike NFAs that process one symbol at a time, GNFAs reads blocks of symbols to go from one state to another.
- The GNFA uses a transition if the corresponding block of symbols belongs to the regular expression of that

Generalized NFA (GNFA)



Generalized NFA (GNFA)

For our mental health, GNFA will satisfy certain conditions

- It has exactly one initial state: q_{start}
- It has exactly one final state: q_{accept}
- No arrow ends on q_{start}, and no arrow starts on q_{accept}
- If there is no arrow from q to s, it means the regular expression there is $\varnothing$

Proof , step 1

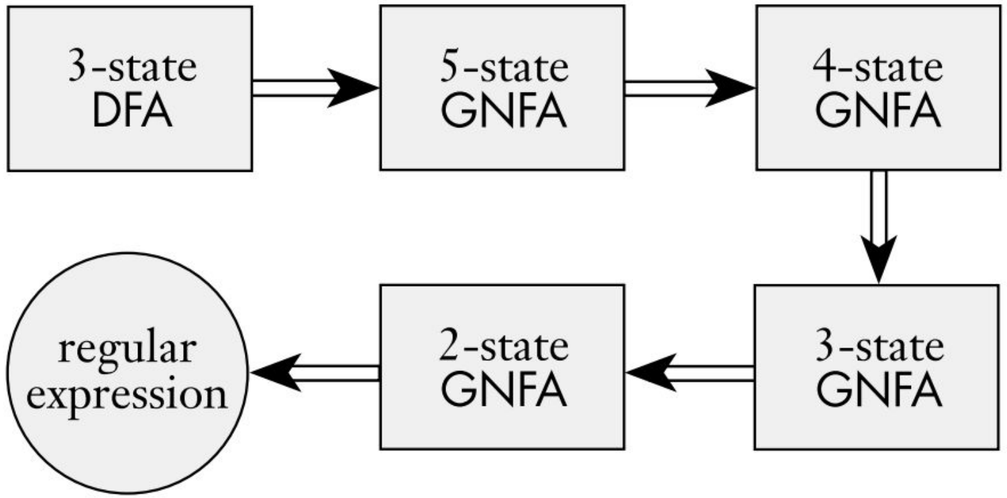
To convert a DFA to a GNFA:

- Add a new initial state q_{start}
- Add ε arrows from q_{start} the states of the DFA
- Add a new accept state q_{accept}
- Add ε arrows from all the states of the DFA to q_{accept}

Proof , step 2

- Convert a GNFA with k states, to a GNFA with k-1 states.
- Do this process while k>2
- When k=2, by definition, there is only one arrow that goes from q_{start} to q_{accept}
- The regular expression on this arrow is the regular expression we are looking for

Proof , step 2

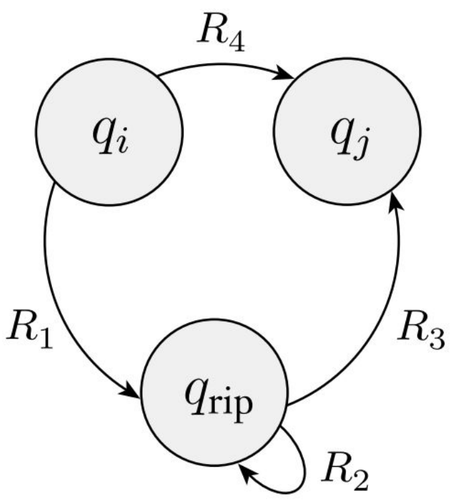

3-state DFA
5-state GNFA
4-state GNFA
3-state GNFA
2-state GNFA
regular expression

Proof , deleting a state

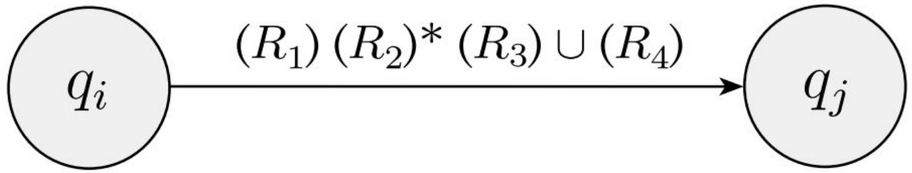
To Convert a GNFA with k states, to a GNFA with k-1 states.

- Pick any state q_{rip}, different from q_{start} and q_{accept}
- For each pair of (not necessarily) states q_i and q_j, we will update the arrow that goes from q_i to q_j such that with the lost computations due to the absence of q_{rip}

Proof , deleting a state



before

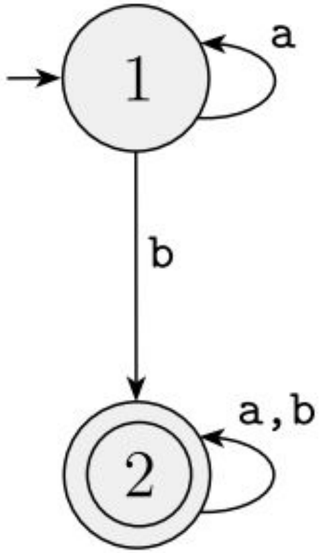


after

Activ

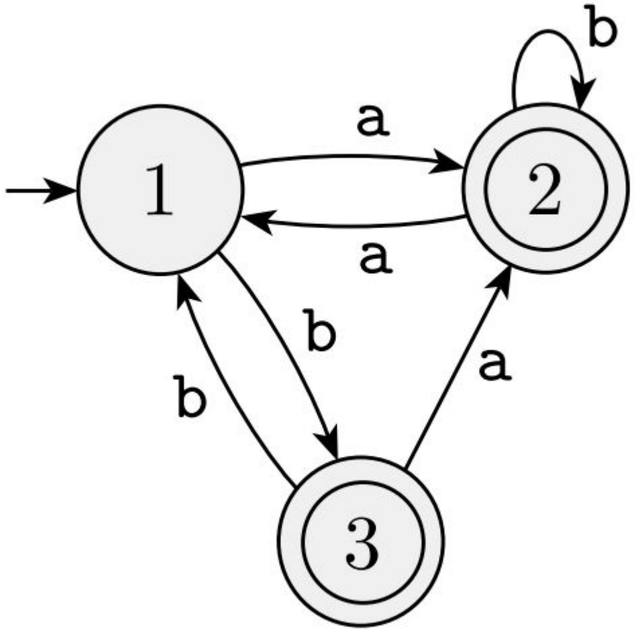
Exercise

Convert the following NFA to a regular expression



Exercise

Convert the following NFA to a regular expression



Cliffhanger

- We saw how powerful was a DFA
- We can build many different Languages
- We can build every finite language, and many infinite languages
- We can build, ANY language....
  - ...
  - ***... or can we?***

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

Thanks

Let’s keep finding mistakes

TRANSFORMATEC